

Java String — String Class & Methods

Question Bank (Basic → Advanced)

A) Basics (foundation) – Theory Questions

Questions 1. What is **String** in Java?

◆ String in Java?

1. Basic Definition

A **String** in Java is a **sequence of characters**.

Example: "Hello", "Java", "12345", "@!#\$%".

In Java, String is not a primitive data type like int or char — it is a **class** from the java.lang package.

So when you write:

```
String s = "Hello";
```

- "Hello" is an **object of class String**.
 - s is a **reference variable** pointing to that String object.
-

2. Internal Representation

- A String is stored as a **character array** internally.
- The class looks like:

```
public final class String implements java.io.Serializable,  
Comparable<String>, CharSequence {  
    private final char value[];  
}
```

- final char[] value → stores characters, cannot be modified after creation (immutability).
-

3. String Literals vs new String()

- **String literal** (stored in String Pool):

```
String s1 = "Java"; // goes to String Pool  
String s2 = "Java"; // s2 points to same object as s1
```

- **Using new keyword** (stored in heap):

```
String s3 = new String("Java"); // new object, not pooled
```

4. Why is String so important?

- Strings are **widely used** in Java for:
 - File paths ("C:\\data\\file.txt")
 - Database URLs ("jdbc:mysql://localhost:3306/db")
 - Network connections ("https://example.com")
 - Messages, user input, etc.
 - Every Java program **heavily depends on Strings**.
-

5. String is Immutable

- Once created, its value **cannot be changed**.
- Example:

```
String s = "Hello";
```

```
s.concat(" World"); // creates new object
System.out.println(s); // prints "Hello"
```

"Hello World" is a new object, "Hello" remains unchanged.

6. Key Features of String

- ✓ **Immutable** – can't be modified once created.
 - ✓ **Stored in String Pool** – saves memory.
 - ✓ **Thread-safe** – safe to use across threads.
 - ✓ **Implements interfaces** – Serializable, Comparable, CharSequence.
 - ✓ **Rich methods** – length(), substring(), concat(), equals(), toUpperCase(), etc.
-

7. Example

```
public class StringExample {
    public static void main(String[] args) {
        String name = "Java";           // String literal
        String lang = new String("Java"); // new String object

        System.out.println(name.toUpperCase()); // JAVA
        System.out.println(name.length());      // 4
        System.out.println(name.charAt(2));     // v
        System.out.println(name.equals(lang));  // true (content same)
        System.out.println(name == lang);      // false (different objects)
    }
}
```

✓ In short:

A String in Java is a sequence of characters, implemented as an immutable object of the String class, stored in memory either in the String Pool or heap, and used everywhere in Java programs.

Questions 2. What makes String **immutable** in Java? What benefits does that give (security, caching, thread safety)?

◆ 1. What does *immutable* mean for Strings?

- **Immutable** means: once a String object is created in memory, its **value cannot be changed**.
- Example:

```
String s = "Hello";  
s.concat(" World");  
System.out.println(s); // prints "Hello", not "Hello World"
```

- Here, `concat(" World")` creates a **new String object** "Hello World".
- The original "Hello" remains unchanged.

So immutability = no method in String modifies the original object. They only return a **new object**.

◆ 2. How is String made immutable internally?

- String is implemented as a **final class**:

```
public final class String implements Serializable,  
Comparable<String>, CharSequence {  
    private final char value[];  
}
```

- **final class** → You cannot extend String and override its behavior.

- private final char[] value → the character array that stores text is final, so its reference cannot be changed after initialization.
 - No method in String changes value[]. Instead, they return a new String.
-

◆ 3. Why did Java designers make Strings immutable?

There are several important reasons:

✓ a) Security

- Strings are used in **sensitive places**:
 - File paths
 - Database URLs
 - Network connections
 - Class loading
- Example:

```
String url = "jdbc:mysql://localhost:3306/db";  
// used to connect DB
```

If strings were mutable, a hacker could change "localhost" to "malicious.com" while the program is running, causing a **security breach**.

Since Strings are immutable, once created, their value is safe.

✓ b) Caching (String Pool)

- Java maintains a **String Constant Pool (SCP)** in the heap.
- When you write:

```
String s1 = "Java";  
String s2 = "Java";
```

- Both s1 and s2 point to the **same object** in the pool.
- This is possible only because Strings are immutable → if they could change, modifying s1 would also affect s2.

- Thus immutability allows **memory optimization** and **reusability** via the string pool.
-

✓ c) Thread Safety

- Strings are **naturally thread-safe** because they cannot be modified.
- Multiple threads can safely share and read the same String without synchronization.

```
String s = "Hello";  
Thread t1 = new Thread() -> System.out.println(s);  
Thread t2 = new Thread() -> System.out.println(s);
```

Both threads see the same "Hello", no risk of corruption.

- If Strings were mutable, you'd need extra locks to prevent concurrent modifications.
-

✓ d) hashCode Caching

- Strings are heavily used as **keys** in **HashMap**, **HashSet**, **Hashtable**.
- Since strings are immutable, their **hashCode** is **cached** after first computation.
 - Example:

```
String key = "abc";  
int hash1 = key.hashCode(); // calculated once  
int hash2 = key.hashCode(); // reused from cache
```

Faster lookups in hash-based collections.

- If Strings were mutable, changing content would break the hashCode contract (equals/hashCode mismatch).
-

✓ e) Performance in Class Loading

- Class names, method names, package names → all represented as String.

- JVM loads classes using these names. If mutable, class references could be hijacked, breaking JVM security.
 - Immutability ensures **consistent behavior of loaded classes**.
-

◆ 4. Benefits Summary

1. **Security** → safe in network, database, file path usage.
 2. **Caching / String Pooling** → saves memory, avoids duplicate objects.
 3. **Thread-Safety** → multiple threads can share strings without synchronization.
 4. **Performance** → hashcode caching improves lookups in collections.
 5. **Reliability** → used in class loading, preventing malicious modifications.
-

✓ In short:

Strings are immutable because of design decisions around **security, memory optimization, and concurrency safety**. This is why Java's core libraries (like String) are trusted and efficient.

Questions 3. == vs equals() for strings—when does each return true?

◆ 1. == operator

- In Java, == checks **reference equality** (i.e., whether two references point to the same object in memory).
- It does **not** check the contents of the object.

Example:

```
String s1 = "Hello";  
String s2 = "Hello";  
System.out.println(s1 == s2); // true
```

✓ Why true?

- Because both "Hello" literals are stored in the **String Constant Pool (SCP)**, and both variables point to the **same memory object**.
-

◆ 2. .equals() method

- In String class, .equals() is **overridden** from Object to check **contents (characters)** of the string.

Example:

```
String s1 = new String("Hello");  
String s2 = new String("Hello");  
  
System.out.println(s1 == s2);    // false (different objects in heap)  
System.out.println(s1.equals(s2)); // true (contents are same: "Hello")
```

◆ 3. When does each return true?

✓ == returns true when:

1. Both references point to the **same object**.

```
String s1 = "Java";  
String s2 = "Java"; // refers to the same SCP object  
System.out.println(s1 == s2); // true
```

2. Or when one reference is assigned to another:

```
String s1 = new String("Java");  
String s2 = s1;  
System.out.println(s1 == s2); // true (both point to same object in heap)
```

✓ .equals() returns true when:

1. The **contents (characters sequence)** are the same, regardless of object location.

```
String s1 = new String("Java");  
String s2 = new String("Java");  
System.out.println(s1.equals(s2)); // true
```

◆ 4. Example comparison table

Code	== Result	.equals() Result	Reason
String s1 = "Java"; String s2 = "Java";	true	true	Both refer to same SCP object, and contents match
String s1 = new String("Java"); String s2 = new String("Java");	false	true	Different heap objects, same content

Code	== Result	.equals() Result	Reason
String s1 = "Java"; String s2 = new String("Java");	false	true	One in SCP, one in heap, same content
String s1 = "Java"; String s2 = "Python";	false	false	Different objects, different content

◆ 5. Important Note

- If you want to compare **content**, always use .equals().
 - Use == only when you specifically need to check whether two references point to the same object (rare in business logic).
-

✓ Quick summary

- == → compares **object reference (memory location)**.
 - .equals() → compares **contents** (characters of the string).
-

👉 Do you want me to also explain why "Java" == "Java" is true but new String("Java") == new String("Java") is false, with a memory diagram of String Pool vs Heap?

Questions 4. How does the String Pool work? When is a string placed there?

◆ 1. What is the String Pool?

- The **String Pool** (or **String Constant Pool, SCP**) is a **special memory region inside the heap** where Java stores **unique string literals**.
 - Its purpose: **save memory** and allow **string reuse**.
-

◆ 2. When is a String placed in the pool?

✓ a) String Literals

Whenever you use **string literals in your code**, they are automatically placed in the String Pool.

```
String s1 = "Java"; // "Java" goes into pool
String s2 = "Java"; // reuses the same "Java" from pool
```

- Both s1 and s2 point to the same object.
-

✓ b) Interning with .intern()

If you create a String object using new, it is placed in the **heap (outside the pool)**.

But you can force it into the pool using .intern().

```
String s1 = new String("Hello"); // Heap
String s2 = s1.intern();         // Pool
String s3 = "Hello";             // Pool
```

```
System.out.println(s2 == s3); // true
```

So `.intern()` ensures that:

- If the string is already in the pool → return that reference.
- If not → add it to the pool and return it.

✓ c) Compile-Time Constants

- Strings created at **compile time** (via literals or final constants) are placed directly in the pool.

```
String s1 = "Ja" + "va"; // compile-time concatenation → "Java" in pool
```

✗ d) Runtime Created Strings

- Strings created at runtime (non-final variables, user input, etc.) are **not automatically placed in the pool**.

```
String a = "Ja";  
String b = "va";  
String s = a + b; // runtime concatenation → new object in heap, NOT in pool
```

If you want it in the pool:

```
s = s.intern(); // manually added
```

◆ 3. How does the pool ensure uniqueness?

- The JVM keeps a **HashSet-like structure** for all pooled strings.
 - Before placing a new string in the pool, JVM checks:
 - If an identical string already exists → reuse it.
 - If not → add it.
 - This guarantees that **no two identical string literals exist in the pool**.
-

◆ 4. Why does the String Pool exist?

1. **Memory efficiency:** Instead of storing "Java" thousands of times, all references point to one copy in the pool.
 2. **Performance:** Comparing strings with `==` works for pooled strings, since they're the same reference.
-

◆ 5. Quick Rules to Remember

- **Literals** → automatically go into the pool.
 - `new String("...")` → object in heap (outside pool).
 - `.intern()` → ensures the string is in the pool.
 - **Compile-time constants** → pooled.
 - **Runtime-created strings** → heap (not pooled unless interned).
-

✓ In short:

The **String Pool** is a JVM-managed cache for strings to save memory and improve performance. A string goes there if it's a **literal, compile-time constant, or explicitly interned.**

Questions 5. What does `new String("abc")` create vs `"abc"`?

◆ 1. `"abc"` (String Literal)

`String s1 = "abc";`

- `"abc"` is a **string literal**.
- JVM automatically places it in the **String Constant Pool (SCP)**.
- If `"abc"` already exists in the pool, JVM **reuses the same object**.
- No new heap object is created; `s1` points to the pooled string.

✓ Memory:

SCP (inside Heap)

`"abc" <--- s1`

◆ 2. `new String("abc")`

`String s2 = new String("abc");`

This does **two things**:

1. The literal `"abc"` is checked/added to the **String Pool** (if not already present).
 2. A **new String object** is created on the **heap**, which **copies the content** of `"abc"` from the pool.
- `s2` points to the **heap object**, not the pooled literal.

✓ Memory:

SCP (inside Heap)

`"abc" // literal`

Heap

[String@1] "abc" <--- s2

- `s2 == "abc" → false` (different objects).
- `s2.equals("abc") → true` (content is same).

◆ 3. Quick Comparison Table

Code	Memory Location	== Result	.equals() Result
<code>String s1 = "abc";</code>	SCP	—	—
<code>String s2 = "abc";</code>	SCP (same as s1)	<code>s1 == s2 → true</code>	true
<code>String s3 = new String("abc");</code>	Heap	<code>s1 == s3 → false</code>	true

◆ 4. Why use `new String()`?

- Rarely needed; usually literals are enough.
- Sometimes used if you **need a distinct object in memory**.
- In most cases, **always prefer literals** → memory-efficient, uses the string pool.

✓ Summary:

- "abc" → placed in **String Pool**, reused.
 - `new String("abc")` → creates a **new heap object** that contains "abc" (pool literal may also exist).
 - Always use `.equals()` to compare content, `==` to check **reference identity**.
-

Questions 6. What does intern() do? When (not) to use it?

Perfect! Let's go deep into **String.intern()**, one of the most subtle but important parts of Java Strings.

◆ 1. What intern() does

The method **String.intern()** does the following:

1. Checks if the string already exists in the **String Pool (SCP)**.
2. If it exists → returns the **reference from the pool**.
3. If it does not exist → adds the string to the pool and returns the reference from the pool.

✓ Essentially, **intern()** ensures that **you get a reference to the canonical (pooled) string object**.

Example:

```
String s1 = new String("Java"); // heap object
String s2 = s1.intern();        // reference from pool
String s3 = "Java";            // literal, pool
```

```
System.out.println(s1 == s2); // false (heap vs pool)
System.out.println(s2 == s3); // true  (both point to pool)
```

◆ 2. Why use intern()?

✓ a) **Save memory**

- When you have **many duplicate strings** created at runtime (e.g., reading from files, user input), **interning** can **reuse existing pool strings** instead of creating new heap objects.


```
String s1 = new String("Hello").intern();  
String s2 = new String("Hello").intern();  
// Both s1 and s2 point to the same pool object
```

✓ b) Reference comparison (==)

- After interning, you can safely use == to compare content because all identical strings point to the **same object in pool**.

```
String a = new String("abc").intern();  
String b = "abc";
```

```
System.out.println(a == b); // true
```

◆ 3. When NOT to use intern()

✗ a) Excessive memory churn

- Pool has **limited memory**, especially in older JVMs.
- Interning **very large or many temporary strings** can increase GC pressure.

✗ b) Unnecessary for literals

- String literals are **already in the pool**, so intern() is redundant.

```
String s1 = "Java";  
String s2 = "Java".intern(); // redundant
```

✗ c) Runtime-created strings with limited reuse

- If a string is **used only once**, interning may actually **waste time** instead of saving memory.
-

◆ 4. Quick rules for intern()

Scenario	Should you use intern()?
String literal	No (already in pool)
Runtime-generated string used multiple times	Yes (saves memory & enables ==)
Temporary string used only once	No (extra overhead)

✓ In short:

- intern(): gives you the canonical pooled reference of a string.
 - Use it to save memory or enable == comparison for runtime strings.
 - Avoid for literals, short-lived, or huge strings that may stress the pool.
-

Questions 7. Is String final? Why does that matter for subclassing/security?

◆ 1. Is String final?

Yes.

```
public final class String implements java.io.Serializable,  
Comparable<String>, CharSequence {  
    //...  
}
```

- final means **you cannot extend the class**.
- So you **cannot create a subclass of String**.

◆ 2. Why does being final matter?

✓ a) Security

Strings are used in **sensitive areas**:

- File paths: "C:/important/file.txt"
- Network connections: "http://localhost:8080"
- Database credentials
- Class names for reflection and class loading

If String were **subclassable**, someone could create a subclass like:

```
class EvilString extends String {  
    // override methods to change behavior  
}
```

- They could **modify the content dynamically** or change hashCode behavior,

- This could lead to **security breaches**, e.g., bypassing file access checks or corrupting data.

✓ Making String final ensures **its behavior is predictable and safe**.

✓ b) Immutability Enforcement

- Strings are **immutable**.
 - If you could subclass String, you could **override methods like `charAt()` or `substring()`** and make a mutable version.
 - That would **break the contract of immutability**, undermining thread safety, caching, and pool usage.
-

✓ c) hashCode Consistency

- Strings are heavily used as **keys in HashMap, HashSet, Hashtable**.
 - Hashcode caching depends on immutability.
 - If you could subclass and override `hashCode()`, the cached value could be inconsistent → **map lookups fail**.
-

✓ d) Thread Safety

- Strings are shared across threads without synchronization because they are immutable.
 - If subclassing were allowed, immutability could be broken → **race conditions**.
-

◆ 3. Summary

Property of String	Reason for being final
Subclassing	Not allowed, ensures security and immutability
Security	Prevents malicious overrides of sensitive string behavior

Property of String	Reason for being final
Immutability	Cannot be broken via subclassing
HashCode caching	Ensures consistent behavior in collections
Thread safety	Safe to share across threads without locks

✓ In short:

- String is final → cannot subclass.
 - This guarantees security, immutability, hashcode consistency, and thread safety.
-

Questions 8. What does "a" + "b" + 1 evaluate to? Why?

Nice one 🦋 This is about **string concatenation vs arithmetic evaluation** in Java.

◆ **Expression:**

"a" + "b" + 1

◆ **Step 1: Evaluate "a" + "b"**

- In Java, the + operator is **overloaded**:
 - For numbers → it does **addition**.
 - If **at least one operand is a String**, it does **string concatenation**.

So:

"a" + "b" → "ab"

◆ **Step 2: Now evaluate "ab" + 1**

- Left side is a String.
- 1 (an int) gets converted to a String using `String.valueOf(1)` internally.

So:

"ab" + 1 → "ab1"

◆ Final Answer

"a" + "b" + 1 → "ab1"

◆ Why not "a" + ("b" + 1) ?

Because Java evaluates + **from left to right** (left-associative).

- If it were "a" + ("b" + 1) → "a" + "b1" → "ab1" (same result here).
- But order matters in other cases:

Example:

1 + 2 + "3"

- 1 + 2 = 3
- "3" + "3" = "33"
- ☞ Result = "33"

But:

"1" + 2 + 3

- "1" + 2 = "12"
- "12" + 3 = "123"
- ☞ Result = "123"

✓ Key takeaway:

- + is left-associative.
 - Once a String is involved, all following + act as **string concatenation**, not numeric addition.
-

Questions 9. Constant folding: why is "a" + "b" equal to "ab" at compile time but x + "b" may not ?

◆ 1. What is Constant Folding?

- Java compiler can **evaluate constant expressions** at compile time and replace them with the result.
- This is an optimization defined in the **Java Language Specification (JLS §15.28)**.

Example:

```
final int x = 10;  
int y = 20 + x; // compiler replaces with int y = 30;
```

Same works for **string concatenation** when operands are **compile-time constants**.

◆ 2. "a" + "b" case

String s = "a" + "b";

- Both "a" and "b" are **string literals** (compile-time constants).
- Compiler folds them into "ab" **before runtime**.
- So in the **.class file**, it's stored as:

String s = "ab";

◆ 3. x + "b" case


```
String x = "a";  
String s = x + "b";
```

- Here x is **not a compile-time constant** (even though it points to "a").
- At compile time, the compiler does **not know** what x will be at runtime (could be "a", could be "z", could even be null).
- So the compiler cannot fold x + "b".
- At runtime, JVM will generate:

```
new StringBuilder().append(x).append("b").toString();
```

◆ 4. Special case: final

If x is declared as a final **compile-time constant**, folding will happen:

```
final String x = "a";  
String s = x + "b"; // compiler replaces with "ab"
```

But if final is assigned dynamically, folding **does not** occur:

```
final String x = new String("a");  
String s = x + "b"; // NOT folded, runtime concatenation
```

◆ 5. Summary Table

Expression	Compile-time folded?	Result stored in .class
"a" + "b"	✓ Yes	"ab"
"a" + "b" + "c"	✓ Yes	"abc"
x + "b"	✗ No (x is variable)	Concatenation at runtime
final String x="a"; x + "b"	✓ Yes	"ab"
final String x=new String("a"); x + "b"	✗ No	Runtime concat

✓ Key takeaway:

- Only compile-time constants are folded.

- String literals (and final constants initialized with literals) are folded.
- Variables, even if they contain the same value, prevent compile-time folding.

Questions 10. hashCode() contract for strings—what does it depend on?

◆ 1. General hashCode() Contract (from Object)

The Java **contract** says:

1. If two objects are **equal according to equals()**, they must return the **same hashCode()**.
2. If two objects are **not equal**, they *may* have the same hashCode() (collision possible).
3. The hashCode() of an object should remain **consistent** during its lifetime (unless fields used in equals() change).

◆ 2. String.hashCode() Implementation

In the String class, hashCode() is **overridden** to ensure that **two equal strings (equals())** always have the same hash.

The formula (from JDK source) is:

$$s[0]*31^{(n-1)} + s[1]*31^{(n-2)} + \dots + s[n-1]$$

where:

- $s[i]$ = Unicode value (UTF-16 code unit) of character at index i .
- n = length of the string.
- 31 = multiplier (chosen because it's an odd prime → fewer collisions, and $31 * i$ can be optimized to $(i \ll 5) - i$).

◆ 3. Example Calculation

```
String str = "abc";  
hashCode = 'a'*31^2 + 'b'*31^1 + 'c'*31^0  
          = 97*961 + 98*31 + 99  
          = 93248 + 3038 + 99  
          = 96385
```

☞ That's exactly what `str.hashCode()` returns.

◆ 4. Properties

- **Depends on characters & order:** "abc" ≠ "cba".
 - **Case-sensitive:** "Abc" ≠ "abc".
 - **Empty string** "" → hash code = 0.
 - **Same content** → **same hash** (even if different objects).
 - **Different content** → **usually different hash**, but collisions are possible.
-

◆ 5. Quick Examples

```
System.out.println("abc".hashCode()); // 96354  
System.out.println("acb".hashCode()); // 96387  
System.out.println("ABC".hashCode()); // 64578  
System.out.println("").hashCode());  // 0
```

```
String s1 = new String("test");  
String s2 = new String("test");  
System.out.println(s1.equals(s2));    // true  
System.out.println(s1.hashCode() == s2.hashCode()); // true
```

◆ 6. Why 31?

- **Prime** → reduces collisions.

- Odd → avoids losing info (if it were even, multiplication might overflow & lose sign).
 - Efficient → $(i * 31)$ can be optimized to $(i \ll 5) - i$.
-

✓ Answer in short:

The hashCode() of a string depends on its sequence of characters and their order, computed using the formula based on a multiplier of 31. Two strings that are equal (equals()) will always have the same hashCode(), as required by the contract.

Questions 11. Do you want me to also explain what happens with **hash collisions** in HashMap when two different strings return the same `hashCode()`?

◆ 1. Quick Recap: HashMap Basics

When you put a key-value pair into a `HashMap<K,V>`:

1. `hashCode()` of the key is computed.
2. That hash is used to find the **bucket index** in the hash table (via $(\text{hash} \& (n-1))$).
3. Keys in the same bucket are stored in a linked list (or tree, in modern JDK).
4. When retrieving (`get(key)`), `hashCode()` finds the bucket, and then `equals()` is used to locate the exact key.

◆ 2. What is a Hash Collision?

A **collision** happens when two different keys produce the **same** `hashCode()`, and therefore map to the **same bucket**.

Example:

```
String s1 = "FB"; // hashCode = 2236
String s2 = "Ea"; // hashCode = 2236
```

```
System.out.println(s1.hashCode()); // 2236
System.out.println(s2.hashCode()); // 2236
```

→ Both "FB" and "Ea" collide in a HashMap.

◆ 3. How HashMap Handles Collisions

- **Pre-Java 8:** All entries in the same bucket are stored in a **linked list**.
 - Lookup becomes $O(n)$ in worst case (all entries collide into one bucket).
 - **Java 8+:** If too many collisions happen in a single bucket (list size > 8), the linked list is converted into a **balanced tree** (Red-Black Tree).
 - Lookup time becomes $O(\log n)$ instead of $O(n)$.
-

◆ 4. Example with Collisions

```
import java.util.*;
```

```
public class CollisionDemo {  
    public static void main(String[] args) {  
        HashMap<String, Integer> map = new HashMap<>();  
  
        map.put("FB", 1);  
        map.put("Ea", 2);  
  
        System.out.println(map.get("FB")); // 1  
        System.out.println(map.get("Ea")); // 2  
    }  
}
```

- "FB".hashCode() == "Ea".hashCode() → same bucket.
 - But equals() differentiates them.
 - So both keys can coexist in the map without overwriting each other.
-

◆ 5. Key Rule

- If two keys have the same `hashCode()` but are not equal (`equals()` returns `false`) → both are stored in the same bucket, resolved via chaining/tree.
 - If two keys are equal (`equals()` returns `true`) → new value replaces the old one.
-

◆ 6. Visual Summary

HashMap Buckets:

[0] -> ("FB", 1) -> ("Ea", 2) // collision, handled by linked list/tree
[1] -> ...

✓ Final Takeaway

- `hashCode()` decides the bucket.
 - `equals()` decides the **exact key** inside that bucket.
 - Collisions don't break HashMap—they just affect performance.
 - Java 8+ improves performance with tree bins for heavily-collided buckets.
-

B) Basics (foundation) –All String Methods

Core queries & basics

- `int length()`
- `boolean isEmpty()`
- `boolean isBlank()` (11)
- `char charAt(int index)`
- `char[] toCharArray()`
- `int hashCode()`
- `String toString()` (returns this)

Perfect! Let's dive **deeply into these core String methods** one by one, with **detailed explanation, syntax, and examples**. I'll also show the subtle differences between them where necessary.

1.int length()

Returns the number of characters in the string.

Syntax:

```
int length = stringVariable.length();
```

Example:

```
public class StringLengthExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
        int len = str.length();  
        System.out.println("Length of '" + str + "' is: " + len);  
    }  
}
```

Output:

Length of 'Hello Java' is: 10

Notes:

- Counts **all characters**, including spaces, special characters, and numbers.
- Always ≥ 0 .

2.boolean isEmpty()

Checks if the string has **0 characters**.

Syntax:

```
boolean result = stringVariable.isEmpty();
```

Example:

```
public class StringIsEmptyExample {  
    public static void main(String[] args) {  
        String str1 = "";  
        String str2 = " ";  
        System.out.println("str1 is empty? " + str1.isEmpty()); // true  
        System.out.println("str2 is empty? " + str2.isEmpty()); // false  
    }  
}
```

```
}
```

Output:

```
str1 is empty? true  
str2 is empty? false
```

Notes:

- Returns true **only if** `length() == 0`.
 - A string containing spaces is **not empty**.
-

3. `boolean isEmpty()` (Java 11+)

Checks if the string is **empty or contains only whitespace characters** (spaces, tabs, newlines).

Syntax:

```
boolean result = stringVariable.isEmpty();
```

Example:

```
public class StringIsEmptyExample {  
    public static void main(String[] args) {  
        String str1 = "";  
        String str2 = " ";  
        String str3 = "Hello";  
  
        System.out.println(str1.isEmpty()); // true  
        System.out.println(str2.isEmpty()); // true  
        System.out.println(str3.isEmpty()); // false  
    }  
}
```

Notes:

- Unlike `isEmpty()`, `isEmpty()` **ignores whitespace**.
 - Useful when checking user input where spaces are not considered valid text.
-

4. `char charAt(int index)`

Returns the **character at a specified index**.

Syntax:

```
char ch = stringVariable.charAt(index);
```

Example:

```
public class CharAtExample {  
    public static void main(String[] args) {  
        String str = "Java";  
        System.out.println("Character at index 0: " + str.charAt(0)); // J  
        System.out.println("Character at index 2: " + str.charAt(2)); // v  
    }  
}
```

Notes:

- Indexing **starts from 0**.
 - Throws `StringIndexOutOfBoundsException` if `index < 0` or `≥ length()`.
-

5. `char[] toCharArray()`

Converts the string into a **character array**.

Syntax:

```
char[] chars = stringVariable.toCharArray();
```

Example:

```
public class ToCharArrayExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
        char[] chars = str.toCharArray();  
        for (char c : chars) {  
            System.out.println(c);  
        }  
    }  
}
```

Output:

```
H  
e  
l  
l  
o
```

Notes:

- Useful when you want to **manipulate individual characters**.
-

6. int hashCode()

Returns the **hash code** of the string (used in hash-based collections like HashMap).

Syntax:

```
int hash = stringVariable.hashCode();
```

Example:

```
public class HashCodeExample {  
    public static void main(String[] args) {  
        String str1 = "Hello";  
        String str2 = "Hello";  
        String str3 = "Java";  
  
        System.out.println("HashCode of str1: " + str1.hashCode());  
        System.out.println("HashCode of str2: " + str2.hashCode()); // same as str1  
        System.out.println("HashCode of str3: " + str3.hashCode());  
    }  
}
```

Output:

```
HashCode of str1: 69609650  
HashCode of str2: 69609650  
HashCode of str3: 2301506
```

Notes:

- Two strings with the **same content** always have the **same hash code**.
 - Useful in **HashMap**, **HashSet** for fast lookup.
-

7. String toString()

Returns the string itself.

Syntax:

String result = stringValue.toString();

Example:

```
public class ToStringExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
        String result = str.toString();  
        System.out.println(result); // Hello Java  
    }  
}
```

Notes:

- For String, toString() just **returns this**.
- In other objects, toString() is used to **represent the object as a string**.

✓ Summary Table:

Method	Returns Type	Description
length()	int	Number of characters
isEmpty()	boolean	true if length == 0
isBlank()	boolean	true if empty or only whitespace
charAt(int)	char	Character at specified index
toCharArray()	char[]	Converts string to char array
hashCode()	int	Returns hash code for string
toString()	String	Returns string itself

Search / index

- int indexOf(int ch)
- int indexOf(int ch, int fromIndex)
- int lastIndexOf(int ch)
- int lastIndexOf(int ch, int fromIndex)
- int indexOf(String str)
- int indexOf(String str, int fromIndex)

- `int lastIndexOf(String str)`
- `int lastIndexOf(String str, int fromIndex)`

Absolutely! Let's go **method by method** for **comparison & ordering** in **Java String class**, with **short explanations** and **small examples** for clarity.

1. `equals(Object obj)`

- **Description:** Checks if two strings have the **exact same content** (case-sensitive).
- **Example:**

```
String str1 = "Java";  
String str2 = "Java";  
String str3 = "java";  
System.out.println(str1.equals(str2)); // true  
System.out.println(str1.equals(str3)); // false
```

2. `equalsIgnoreCase(String anotherString)`

- **Description:** Compares two strings **ignoring case**.
- **Example:**

```
String str1 = "Java";  
String str2 = "java";  
System.out.println(str1.equalsIgnoreCase(str2)); // true
```

3. `compareTo(String anotherString)`

- **Description:** Compares two strings **lexicographically** (dictionary order).
- Returns **0** if equal, **negative** if first < second, **positive** if first > second.
- **Example:**

```
String str1 = "Apple";  
String str2 = "Banana";  
System.out.println(str1.compareTo(str2)); // negative  
System.out.println(str2.compareTo(str1)); // positive  
System.out.println(str1.compareTo("Apple")); // 0
```

4. compareToIgnoreCase(String str)

- **Description:** Same as compareTo, but **ignores case**.
- **Example:**

```
String str1 = "apple";  
String str2 = "Apple";  
System.out.println(str1.compareToIgnoreCase(str2)); // 0
```

5. contentEquals(CharSequence cs)

- **Description:** Checks if string content **matches a CharSequence** (e.g., StringBuilder, StringBuffer).
- **Example:**

```
String str = "Hello";  
StringBuilder sb = new StringBuilder("Hello");  
System.out.println(str.contentEquals(sb)); // true
```

6. contentEquals(StringBuffer sb)

- **Description:** Checks if string content **matches a StringBuffer exactly**.
- **Example:**

```
String str = "World";  
StringBuffer sb = new StringBuffer("World");  
System.out.println(str.contentEquals(sb)); // true
```

7. regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)

- **Description:** Checks if a **region of one string matches another string's region**.
- ignoreCase = true → ignores case.
- **Example:**

```
String str1 = "HelloJava";  
String str2 = "java";  
System.out.println(str1.regionMatches(true, 5, str2, 0, 4)); // true
```

8. startsWith(String prefix)

- **Description:** Checks if string **begins with a specified prefix**.
- **Example:**

```
String str = "HelloJava";
```

```
System.out.println(str.startsWith("Hello")); // true
System.out.println(str.startsWith("Java")); // false
```

9. startsWith(String prefix, int toffset)

- **Description:** Checks if substring starting at toffset **starts with prefix**.
- **Example:**

```
String str = "HelloJava";
System.out.println(str.startsWith("Java", 5)); // true
```

10. endsWith(String suffix)

- **Description:** Checks if string **ends with a specified suffix**.
- **Example:**

```
String str = "HelloJava";
System.out.println(str.endsWith("Java")); // true
System.out.println(str.endsWith("Hello")); // false
```

11. contains(CharSequence s)

- **Description:** Checks if string **contains the specified sequence** anywhere.
- **Example:**

```
String str = "HelloJava";
System.out.println(str.contains("Java")); // true
System.out.println(str.contains("World")); // false
```

✓ Summary Table for Quick Reference:

Method	Description
equals	Case-sensitive equality
equalsIgnoreCase	Case-insensitive equality
compareTo	Lexicographical comparison
compareToIgnoreCase	Lexicographical, ignore case
contentEquals(CharSequence)	Matches content with CharSequence
contentEquals(StringBuffer)	Matches content with StringBuffer
regionMatches	Compare regions of two strings
startsWith(prefix)	Check beginning of string
startsWith(prefix, offset)	Check substring from offset
endsWith(suffix)	Check end of string

Method	Description
contains	Check if string contains a sequence

Substrings & sequences

- `String substring(int beginIndex)`
- `String substring(int beginIndex, int endIndex)`
- `CharSequence subSequence(int beginIndex, int endIndex)`

Absolutely! Let's look at the **substring** and **sequence methods** in Java's `String` class with explanation and small examples.

1. `String substring(int beginIndex)`

- **Description:** Returns a new string starting from `beginIndex` to the end.
- **Indexing starts at 0.**

Example:

```
public class SubstringExample1 {  
    public static void main(String[] args) {  
        String str = "HelloJava";  
        String sub = str.substring(5); // from index 5 to end  
        System.out.println(sub); // Output: Java  
    }  
}
```

Notes:

- Throws `StringIndexOutOfBoundsException` if `beginIndex < 0` or `beginIndex > length()`.
-

2. `String substring(int beginIndex, int endIndex)`

- **Description:** Returns a **new string** from `beginIndex` to `endIndex-1` (`endIndex` is exclusive).

Example:

```
public class SubstringExample2 {  
    public static void main(String[] args) {  
        String str = "HelloJava";  
        String sub = str.substring(0, 5); // from index 0 to 4  
        System.out.println(sub); // Output: Hello  
    }  
}
```

Notes:

- `beginIndex` must be $\leq$ `endIndex`.
 - Throws `StringIndexOutOfBoundsException` if indices are invalid.
-

3. `CharSequence subSequence(int beginIndex, int endIndex)`

- **Description:** Returns a **character sequence** (`CharSequence` interface) from `beginIndex` to `endIndex-1`.
- Functionally similar to `substring()`, but returns **`CharSequence`**, not `String`.

Example:

```
public class SubSequenceExample {  
    public static void main(String[] args) {  
        String str = "HelloJava";  
        CharSequence seq = str.subSequence(0, 5);  
        System.out.println(seq); // Output: Hello  
    }  
}
```

Notes:

- Useful when a method requires a `CharSequence` rather than `String`.

- Can be cast to String if needed: `String s = seq.toString();`

✓ Quick Comparison

Method	Returns	Range
<code>substring(int beginIndex)</code>	String	From beginIndex to end
<code>substring(int beginIndex, int endIndex)</code>	String	From beginIndex to endIndex-1
<code>subSequence(int beginIndex, int endIndex)</code>	CharSequence	From beginIndex to endIndex-1

Case conversion

- `String toLowerCase()`
- `String toLowerCase(Locale locale)`
- `String toUpperCase()`
- `String toUpperCase(Locale locale)`

Sure! Let's go through **Java String case conversion methods** with clear explanations and examples.

1. `String toLowerCase()`

- **Description:** Converts all characters of the string to **lowercase** based on **default locale**.
- **Syntax:**

```
String lower = stringVariable.toLowerCase();
```

Example:

```
public class ToLowerCaseExample {  
    public static void main(String[] args) {
```

```
String str = "Hello Java";
String lower = str.toLowerCase();
System.out.println(lower); // Output: hello java
}
}
```

2. String toLowerCase(Locale locale)

- **Description:** Converts all characters to lowercase according to a **specific locale**. Useful for language-specific rules.
- **Example:**

```
import java.util.Locale;

public class ToLowerCaseLocaleExample {
    public static void main(String[] args) {
        String str = "İstanbul"; // Turkish capital I with dot
        String lower = str.toLowerCase(new Locale("tr", "TR"));
        System.out.println(lower); // Output: istanbul
    }
}
```

Notes:

- Different locales may treat letters differently (like Turkish “İ” vs “i”).

3. String toUpperCase()

- **Description:** Converts all characters of the string to **uppercase** based on **default locale**.
- **Syntax:**

```
String upper = stringVariable.toUpperCase();
```

Example:

```
public class ToUpperCaseExample {
    public static void main(String[] args) {
        String str = "Hello Java";
        String upper = str.toUpperCase();
        System.out.println(upper); // Output: HELLO JAVA
    }
}
```

4. String toUpperCase(Locale locale)

- **Description:** Converts all characters to uppercase according to a **specific locale**.
- **Example:**

```
import java.util.Locale;
```

```
public class ToUpperCaseLocaleExample {  
    public static void main(String[] args) {  
        String str = "istanbul"; // lowercase  
        String upper = str.toUpperCase(new Locale("tr", "TR"));  
        System.out.println(upper); // Output: İSTANBUL  
    }  
}
```

✓ Summary Table

Method	Description	Locale Dependent?
toLowerCase()	Convert all chars to lowercase	Uses default locale
toLowerCase(Locale)	Convert all chars to lowercase	Uses specified locale
toUpperCase()	Convert all chars to uppercase	Uses default locale
toUpperCase(Locale)	Convert all chars to uppercase	Uses specified locale

Trimming / whitespace utilities

- String trim()
- String strip() (11)
- String stripLeading() (11)
- String stripTrailing() (11)
- String stripIndent() (15)
- String indent(int n) (12) // positive adds, negative removes common indentation

Absolutely! Let's go through **Java String trimming and whitespace utility methods**, including the **new methods** introduced in **Java 11 and 15**, with explanations and examples.

1. String trim()

- **Description:** Removes **leading and trailing whitespace** (spaces, tabs, newlines) from a string.
- **Example:**

```
public class TrimExample {  
    public static void main(String[] args) {  
        String str = "  Hello Java  ";  
        System.out.println("Before trim: " + str + "");  
        System.out.println("After trim: " + str.trim() + "");  
    }  
}
```

Output:

```
Before trim: '  Hello Java  '  
After trim: 'Hello Java'
```

2. String strip() (Java 11)

- **Description:** Similar to trim(), but uses **Unicode-aware whitespace removal**.
- **Example:**

```
public class StripExample {  
    public static void main(String[] args) {  
        String str = "\u2002Hello Java\u2002"; // \u2002 is Unicode space  
        System.out.println("Before strip: " + str + "");  
        System.out.println("After strip: " + str.strip() + "");  
    }  
}
```

3. String stripLeading() (Java 11)

- **Description:** Removes **whitespace only from the beginning** of the string.
- **Example:**

```
public class StripLeadingExample {
```

```
public static void main(String[] args) {  
    String str = " Hello Java ";  
    System.out.println("'" + str.stripLeading() + "'"); // 'Hello Java '  
}  
}
```

4. String stripTrailing() (Java 11)

- **Description:** Removes **whitespace only** from the end of the string.
- **Example:**

```
public class StripTrailingExample {  
    public static void main(String[] args) {  
        String str = " Hello Java ";  
        System.out.println("'" + str.stripTrailing() + "'"); // ' Hello Java'  
    }  
}
```

5. String stripIndent() (Java 15)

- **Description:** Removes **common leading indentation** from **multiline strings**. Useful for text blocks.
- **Example:**

```
public class StripIndentExample {  
    public static void main(String[] args) {  
        String str = " line1\n line2\n line3";  
        System.out.println("Before stripIndent:\n" + str);  
        System.out.println("After stripIndent:\n" + str.stripIndent());  
    }  
}
```

Output:

Before stripIndent:

```
line1  
line2  
line3
```

After stripIndent:

```
line1  
line2  
line3
```

6. String indent(int n) (Java 12)

- **Description:** Adds or removes **common indentation** for **multiline strings**.
 - Positive n → adds spaces at the start of each line.
 - Negative n → removes spaces at the start of each line (up to n).
- **Example:**

```
public class IndentExample {
    public static void main(String[] args) {
        String str = "line1\n line2\n  line3";

        System.out.println("Original:\n" + str);
        System.out.println("Indent +2:\n" + str.indent(2));
        System.out.println("Indent -1:\n" + str.indent(-1));
    }
}
```

✓ Summary Table

Method	Description	Introduced in
trim()	Removes leading & trailing whitespace	Java 1.0
strip()	Unicode-aware trim	Java 11
stripLeading()	Removes leading whitespace only	Java 11
stripTrailing()	Removes trailing whitespace only	Java 11
stripIndent()	Removes common indentation in multiline strings	Java 15
indent(int n)	Adds/removes indentation in multiline strings	Java 12
